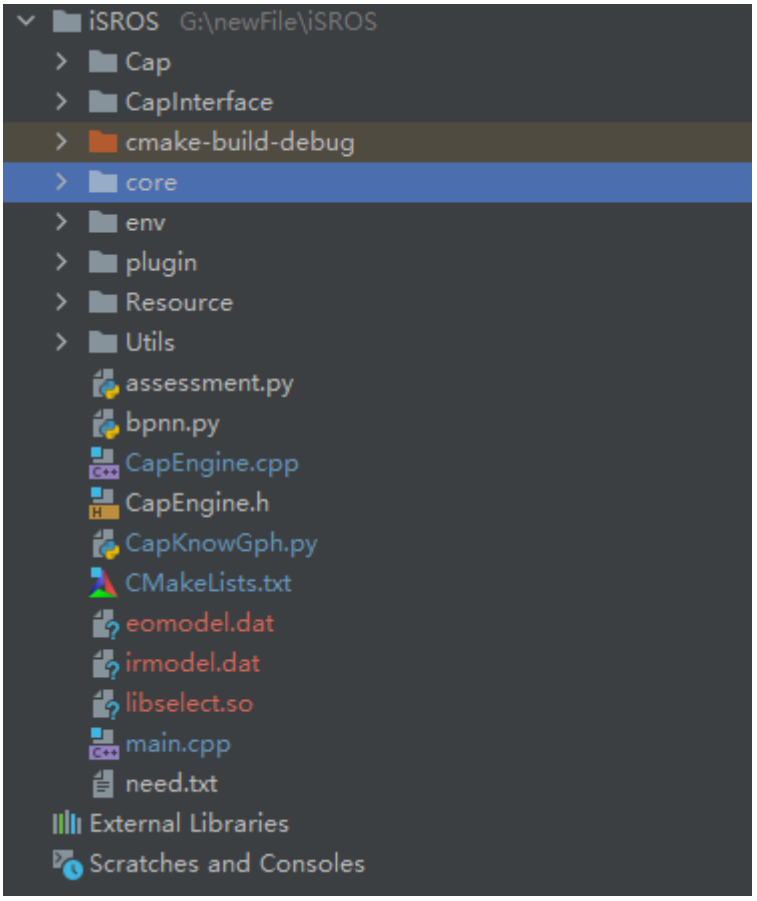
系统的结构说明看论文，这里讲一下目录结构和一些可能涉及修改的地方
服务器账号zyzhu
密码yangzhe
建议各位先克隆代码，然后在自己的用户目录下去开发调试包括run

目录结构和相关说明



Cap是能力的数据结构，一般来说不需要修改
CapInterface是能力访问接口，有可能修改
Core是几个核心子系统，涉及ip可能会改的频繁
env是环境数据的结构，一般来说不需要修改
plugin是插件，包括插件管理器和api，一般来说不需要修改除非新增插件或者修改插件api
resource是资源（即无人机），一般来说不需要修改
Util是一些工具类和函数，包括全局变量等，可能会改
CapEnginee.cpp是能力引擎，一般来说不需要修改
CapKnowGraph.py是访问neo4j图数据库的python函数，可能会改
.dat是李春鸽的模型，自行替换
need.txt是能力需求，自行修改，但格式如果变了需要改代码CapNeedManager.cpp
以上的这个目录结构是开发环境，实际部署的时候会有.so插件，这个在服务器（10.68.121.195的机器）上有，如下所示，我目前把这些插件都放在仓库的“历史插件”目中下

```
Last login: Fri Mar  3 14:13:47 2023 from 10.71.47.45
zyzhu@hcps-uavs:~$ ls
accessplugin  Desktop     Downloads  iSSROS       khelper  Pictures  selectPlugin  texlive.iso        Videos
assessPlugin  Documents   iROS       iSSROS_0205  Music    Public    Templates     thinclient_drives
zyzhu@hcps-uavs:~$ cd iSSROS
zyzhu@hcps-uavs:~/iSSROS$ ls
assessment.py  CapEngine.cpp  CapKnowGph.py      core         irmodel.dat        main.cpp  __pycache__
bpnn.py        CapEngine.h    cmake-build-debug  env          libAssessPlugin.so need.txt  Resource
Cap            CapInterface   CMakeLists.txt     eomodel.dat  libselect.so       plugin    Utils
zyzhu@hcps-uavs:~/iSSROS$
```

Cap

Capability是能力类，对应的是图数据库里面的能力，属性是一一对应起来的
CapabilityAllocated是能力分配表，封装了map，就是需求id和可用能力id的映射
CapabilityCanUse是可用能力表，封装了能力id，评估的值level，位置position和功耗power
CapabilityNeed是能力需求表，封装了一个map记录需求id和需求的自然语言描述，和一个vector，记录了需求的阈值（这个是跟可用能力需要被评估的指标是对应的）

CapInterface

CapInterface就是访问图数据库的cpp接口，主要有两个，获取能力（在CapEngine.cpp中调用）和获取资源（在CapDispatchManager.cpp中调用）

```
public:
    vector<Capability * >getCapability(const char* FileName,const char* FuncName,const char* CapName) const;
    Resource* getResource(const char* FuncName,const int capId)const;
};
```

获取能力getCapability（python接口文件，python函数名称，能力名称）：

```
    resourceCapTB = CIM->getCapability("CapKnowGph", "queryByLabel", capName);
```

获取资源getResource（python函数名称，能力id）

```
    resources.push_back(CIM->getResource("queryResourceById",iter->second));
```

CapKnowGph.py

设置连接

```
    driver = GraphDatabase.driver("neo4j://10.68.160.21:7687",
                                  auth=("neo4j", "zhangyu123"))
```

通过能力的label查询原子能力，返回dict数组

```
    def getByLabel(tx,label):
        query = ("MATCH (n)<-[:ns0__isA]-(m) where any(label in labels(n) WHERE label in
    [$message]) RETURN m")

        res = tx.run(query,message=label).data()
        return res
```

上面这个就是写cql语句的地方，根据neo4j提供的cql语法来写

查询上面的cql已经写了，这里主要是连接数据库查询数据和数据的处理，就是规范一下数据格式，以list的形式返回（因为上面的代码返回的res是个dict，不太好处理）。另外，如果你变了数据库，这里也要修改数据库名字database="capability0206"

```
# 封装getByLabel，返回处理好的数据——list
def queryByLabel(label):
    with driver.session(database="capability0206") as session:
        tx = session.begin_transaction()
        res = []
        res = getByLabel(tx, "ns0__"+label)
        tx.commit()
        tx.close()

        list = []
        for i in range(len(res)):
            list.append(res[i]['m'])
        properties = res[0]['m']
        return list
```

如果你要新加一些api，一般按照两步，第一步写cql，第二步执行cql并处理返回结果。可以模仿上面的例子。

Core

CapAdaptiveManager.cpp

这个就是自适应模块，用来调用评估匹配插件的，基本行行都是注释，自己就看得懂了

CapDispatchManager.cpp

根据能力分配表去发送指令采集图像
需要先查图数据库，找到能力对应的资源，然后再给airsim发结果数据。所以需要涉及访问知识图和airsim。
airsim的ip变了要来这里修改

CapNeedManager.cpp

构建能力需求表，需要解析需求的txt文件，所以涉及文件读写

ContextRenewManager.cpp

获取环境，无人机位置数据，主要是udp的接收代码
环境数据的格式应该是：时间#亮度#雾#雨#沙尘
位置数据格式是：时间#N#E#D

Utils

Global.cpp主要是定义了能力表等全局变量
Single_T.cpp是单例模式的模板
Utils.cpp主要是一些工具函数的封装（获取能力的基础字段，获取python函数的部分代码，字符串切割）

环境

cmake

```
cmake_minimum_required(VERSION 3.0)
project(IROS)

set(CMAKE_CXX_STANDARD 14)
SET(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -std=c++11 -pthread")

#include_directories(/usr/include/python3.8)
#link_directories(/usr/lib/python3.8/config-3.8-x86_64-linux-gnu)
include_directories(/usr/include/python3.10)
link_directories(/usr/lib/python3.10/config-3.10-x86_64-linux-gnu)
#link_directories(/tmp/IROS/libs)
#link_directories(/home/zyzhu/IROS/libs)
link_directories(/home/zyzhu/iSSROS/libs)

add_executable(IROS main.cpp Cap/Capability.cpp Cap/Capability.h Cap/Parameter

#target_link_libraries(IROS libpython3.8.so)
target_link_libraries(IROS libpython3.10.so)
TARGET_LINK_LIBRARIES(IROS dl)
```

这是目前的cmakelist，用的是python10的环境，服务器上装的就是这个，切换python环境的话，需要修改上图相应的注释。修改之后需要修改头文件，如CapInterface.cpp等，冒红不用管，能跑就行。

```
5   #include "CapInterfaceManager.h"
6   //#include <Python.h>
7   #include <python3.10/Python.h>
8   //#include <python3.8/Python.h>
9   #include "../Utils/Utils.h"
```

具体需要修改的代码，我列一下：

CMakeLists.txt

CapInterface.cpp

Utils.h

assess目录下的插件相关的代码也要改（http://10.68.21.210:8888/harold/iSSROS/src/master/code/assess）

涉及ip的代码：

CapDispatchManager.cpp

airsimCode.py（http://10.68.21.210:8888/harold/iSSROS/src/master/code/AirSim/airsimCode.py）

python包

如果需要用新的包，直接pip安装就行，目前的包在下面的路径

```
zyzhu@hcps-uavs:~/.local/lib/python3.10/site-packages$ ls
joblib                  neo4j                  numpy                  numpy-1.24.2.dist-info  testkitbackend
joblib-1.2.0.dist-info  neo4j-5.2.1.dist-info  numpy-1.23.5.dist-info  numpy.libs
zyzhu@hcps-uavs:~/.local/lib/python3.10/site-packages$ pwd
/home/zyzhu/.local/lib/python3.10/site-packages
zyzhu@hcps-uavs:~/.local/lib/python3.10/site-packages$
```

插件

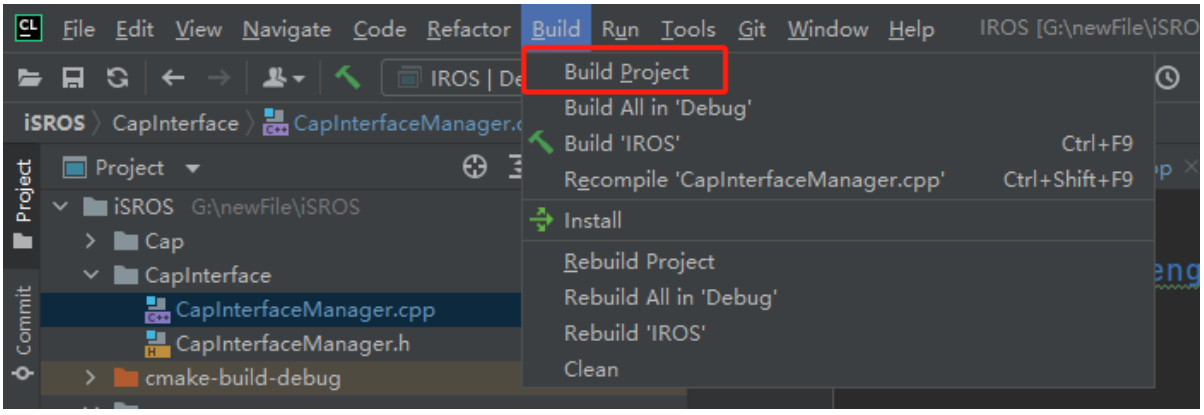
关于插件的代码，是文件夹下的access和select两个包，你们需要修改评估或者匹配的函数的话，只用修改AssessPlugin.cpp里面的这个函数

```
// 实现Demo中定义的虚函数
bool AssessPlugin::assess(vector<Capability*> &capabilities,Environment* curEnv){
//    评估
    map<string,string> models;
    models["EOCap"] = "eoAssess";
    models["IRCap"] = "irAssess";
//    models["RGBCap2"] = "eoAssess";
//    models["IRCap2"] = "irAssess";
```

接口就是bool assess（vector...）

select一样的，代码自己问端家鑫，这里没有。只有评估的代码，模型不是最新的，最新的.dat模型找李春鸽，替换一下就行。

插件函数修改完毕之后，需要build一下



build完了之后，服务器的目录下就会有.so了

```
zyzhu@hcps-uavs:~/assessPlugin/cmake-build-debug$ ls
CMakeCache.txt  CMakeFiles  cmake_install.cmake  DemoPlugin.cbp  libAssessPlugin.so  Makefile
zyzhu@hcps-uavs:~/assessPlugin/cmake-build-debug$ pwd
/home/zyzhu/assessPlugin/cmake-build-debug
zyzhu@hcps-uavs:~/assessPlugin/cmake-build-debug$
```

把这个.so复制到iSSROS目录下就可以run了。（当然你们也可以在iSSROS下建一个libs目录，里面存放所有的.so，这样的话也要改CapAdaptiveManager.cpp代码第26行PluginHandler ph = pm->LoadLibrary("../libAssessPlugin.so");）

```
24      // 插件管理器加载libdemoplugin.so
25  //    PluginHandler ph = pm->LoadLibrary("../lib/libdemoplugin.so");
26      PluginHandler ph = pm->LoadLibrary("../libAssessPlugin.so");
```